Project Design Phase-II

Solution Requirements (Functional & Non-functional)

Date	7 march 2026
Team ID	SWTID-2026-2725
Project Name	SpendWise
Maximum Marks	4 Marks

Functional Requirements: (Flow Diagram Style)

```
+-----------------+
|   User Signup   |
| Name, Email,    |
| Password        |
+--------+---------+
         |
         v
+-----------------+
|   User Login    |
```

```
| Email & Password |
+--------+---------+
|
v
+-----------------+
| Authentication  |
| JWT Token       |
| Generated       |
+--------+---------+
|
v
+-----------------+
| Expense Manager |
+--------+---------+
|
-----------------------------------
|          |        |     |
v          v        v     v
+-------+   +-------+  +-------+ +-------+
| Add   |   | View  |  | Edit  | |Delete |
|Expense|   |Expense|  |Expense| |Expense|
+-------+   +-------+  +-------+ +-------+

|
```

```
v
+-----------------+
| Expense Reports |
| Daily / Monthly |
| Spending Summary |
+-----------------+

|
v
+-----------------+
| Database Storage |
| MongoDB          |
+-----------------+
```

Non-Functional Requirements (Diagram Style):

```
+----------------------+
|    Usability         |
| Simple Interface     |
| Easy Navigation      |
+-----------+----------+

+----------------------+
|    Security          |
```

| JWT Authentication |

| Password Encryption |

+----------+----------+

+----------+

| Performance |

| Fast API Response |

| Efficient Data Fetch |

+----------+----------+

+----------+

| Reliability |

| Stable System |

| Error Handling |

+----------+----------+

+----------+----------+

| Scalability |

| Can handle many users |

+----------+----------+

SpendWise – Explanation of Requirements:

A. Functional Requirements (Explanation)

Functional requirements describe the main functions or operations that the SpendWise system must perform.

✓ **User Registration**

The user must be able to create a new account in the SpendWise application. During registration, the user provides details such as **name, email address, and password**. This

information is stored securely in the database so that the user can access the system later.

✓ **User Login**

After registration, the user can log in to the application using their **email and password**. The system verifies the login details before allowing the user to enter the application.

✓ **Authentication**

Once the login is successful, the system generates a **JWT (JSON Web Token)** for the user. This token is used to verify the identity of the user and allows them to access the protected features of the application securely.

✓ **Expense Management**

SpendWise allows users to **manage their daily expenses**. The user can perform the following operations:

- **Add Expense:** Users can enter new expenses such as food, transport, shopping, etc.
- **View Expense:** Users can see the list of all their recorded expenses.

- **Edit Expense:** Users can update or modify previously entered expenses.
- **Delete Expense:** Users can remove any incorrect or unnecessary expense record.

These features help users keep track of their spending effectively.

B. Non-Functional Requirements (Explanation)

Non-functional requirements describe the **quality and performance characteristics** of the SpendWise system.

✓ **Usability**

The SpendWise application should have a **simple and user-friendly interface**. The design must be easy to understand so that beginners can use the application without difficulty.

✓ **Security**

Security is very important in the system. All protected APIs must use **JWT authentication**. User passwords must be stored in an **encrypted (hashed) format** to prevent unauthorized access.

✓ **Performance**

The application should respond quickly to user requests. The **API response time should be fast**, even when many users are accessing the system at the same time.

✓ **Reliability**

The system must work **consistently without errors or crashes**. It should properly handle both valid and invalid user requests to ensure smooth operation.

- ✓ **Scalability**

The system should be able to **handle an increasing number of users and transactions** in the future without affecting its performance.